

EXPERIMENT – 8

Name: K ANUSRI

Roll No: 2520030432

Section: 8

Title

Dynamic Memory Allocation using malloc(), calloc(), realloc(), free() and Memory Leak Detection using Valgrind

Aim

To develop a program using malloc(), calloc(), realloc(), and free(), monitor memory allocation behavior, and identify memory leaks using Valgrind.

Theory

- **malloc()** – Allocates a specified amount of memory dynamically.
- **calloc()** – Allocates memory for multiple elements and initializes it to zero.
- **realloc()** – Changes the size of previously allocated memory.
- **free()** – Releases dynamically allocated memory.
- **Valgrind** – A memory debugging tool used to detect memory leaks and memory-related errors.

Algorithm

1. Start the program.
2. Allocate memory using malloc().
3. Allocate memory using calloc().
4. Resize the allocated memory using realloc().
5. Release the allocated memory using free().
6. Leave one memory block unfreed to demonstrate memory leak detection.
7. Compile the program with debugging information.
8. Execute the program normally.
9. Run the program using Valgrind with leak checking enabled.
10. Observe the memory leak information reported by Valgrind.
11. Stop the program.

Program Code

```
k_anusri@DESKTOP-TM20BAK x + v
k_anusri@DESKTOP-TM20BAQ:~$
k_anusri@DESKTOP-TM20BAQ:~$ cat > exp8_memory.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a = malloc(5 * sizeof(int));
    int *b = calloc(5, sizeof(int));
    int *leak = malloc(100 * sizeof(int));

    if (a == NULL || b == NULL || leak == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        a[i] = i + 1;

    a = realloc(a, 10 * sizeof(int));

    if (a == NULL)
    {
        printf("Reallocation failed.\n");
        free(b);
        free(leak);
        return 1;
    }

    printf("malloc() allocation successful.\n");
    printf("calloc() allocation successful.\n");
    printf("realloc() allocation successful.\n");

    free(a);
    free(b);

    printf("free() completed.\n");
    printf("One allocated block is intentionally not freed for Valgrind leak detection.\n");

    return 0;
}
EOF
```

Execution Screenshots

Screenshot 1 – Compilation and Program Execution

```
k_anusri@DESKTOP-TM20BAQ:~$ gcc -g exp8_memory.c -o exp8_memory
k_anusri@DESKTOP-TM20BAQ:~$ ./exp8_memory
malloc() allocation successful.
calloc() allocation successful.
realloc() allocation successful.
free() completed.
One allocated block is intentionally not freed for Valgrind leak detection.
```

Screenshot 2 – Valgrind Memory Leak Detection

```
k_anusri@DESKTOP-TM20BAQ:~$ valgrind --leak-check=full ./exp8_memory
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.26.0-0ubuntu1
See 'snap info valgrind' for additional versions.
```

Result

The program successfully demonstrated dynamic memory allocation using `malloc()`, `calloc()`, `realloc()`, and `free()`. Memory leak detection was successfully performed using Valgrind.